

WS_ENDPOINT_AUTH_DESIGN

/ws WebSocket Endpoint — Authentication Design (DESIGN ONLY, no code change)

Revision	Created	Last modified	Status
1	2026-07-08	2026-07-08	draft

Status summary: Design proposal for closing a confirmed unauthenticated-access gap on the /ws WebSocket endpoint. Awaits operator decision (§11.4.66) before any implementation work starts. No code touched by this document.

Issues / Fixed: none opened yet — this doc is the pre-work research + design artefact per §11.4.150 (deep research before fix) that a follow-up workable item will cite when the operator approves an option.

Table of contents

- [1. Confirmed finding: /ws is genuinely unauthenticated today](#)
- [2. What /ws actually exposes](#)
- [3. Why this is a real, if partially bounded, gap](#)
- [4. The core constraint: browsers can't send Authorization on a WS handshake](#)
- [5. Research: standard patterns for browser WebSocket auth](#)
- [6. Recommended primary option + implementation sketch](#)
- [7. Regression-guard test plan \(§11.4.135\) + RED plan \(§11.4.115\)](#)
- [8. Operator decision \(§11.4.66\)](#)
- [9. Honest concerns / UNCONFIRMED items](#)
- [Sources verified](#)

1. Confirmed finding: /ws is genuinely unauthenticated today

File: helix_code/internal/server/server.go

```
430:      s.router.GET("/ws", s.handleWebSocket)
```

registered with **zero middleware** — contrast with the two DZ-05-hardened wire routes three lines above it:

```
426:    s.router.POST("/v1/chat/completions", s.wireFacadeAuthMiddleware())
427:    s.router.POST("/v1/messages", s.wireFacadeAuthMiddleware(), s.authMiddleware())
```

The handler itself (server.go:623-625):

```
func (s *Server) handleWebSocket(c *gin.Context) {
    s.mcp.HandleWebSocket(c.Writer, c.Request)
}
```

delegates straight into internal/mcp/server.go's MCPServer.HandleWebSocket (internal/mcp/server.go:142), which upgrades the connection via gorilla/websocket with:

```
// internal/mcp/server.go:100-112
func NewMCPServer() *MCPServer {
    return &MCPServer{
        upgrader: websocket.Upgrader{
            CheckOrigin: func(r *http.Request) bool {
                // In production, you should validate the origin
                return true
            },
        },
        ...
    }
}
```

CheckOrigin unconditionally returns true — literally every origin is accepted, and the in-code comment is itself an admission this was never finished. There is no auth check anywhere between router.GET("/ws", ...) and the Upgrade() call. **Confirmed: any client, from any origin, with no credential, can complete the WS handshake against /ws today.**

server.go:87 constructs the mcp.MCPServer with mcp.NewMCPServer() and never wires an auth check on top of it before registering the route — so this is not a case of auth living elsewhere in the stack; it genuinely does not exist for this path.

2. What /ws actually exposes

The MCP session speaks a small JSON-RPC-shaped protocol (internal/mcp/server.go:194-212, handleMessage):

method	Handler
initialize	handleInitialize

method	Handler
tools/list	handleListTools
tools/call	handleCallTool
notifications/capabilities	handleCapabilities
ping	handlePing

tools/call dispatches into `s.tools[name].Handler` — an arbitrary `ToolHandler func(ctx, session, args) (interface{}, error)` — i.e. this is a **capability-execution surface**, not merely an LLM token stream. That is a materially bigger blast radius than the two wire-facade routes DZ-05 already closed, which only drive `provider.Generate/GenerateStream`.

As-shipped mitigating fact (verified, not assumed): grepping every non-test .go file under `helix_code/` for calls to `(*mcp.MCPServer).RegisterTool` turns up exactly one call site — `internal/mcp/doc.go:30`, a doc-comment *example*, not executable wiring. `server.go:87's mcp.NewMCPServer()` never has `RegisterTool` called on it anywhere in the production construction path. **Today, tools/list returns an empty list and tools/call always fails with -32601 Tool not found.** The capability-execution risk is currently latent, not live.

3. Why this is a real, if partially bounded, gap

Even with zero tools registered, the unauthenticated + any-origin /ws is a live defect on three independent axes:

1. **Resource exhaustion / DoS.** Every successful handshake spawns a goroutine (`go s.handleSession(session)`, `server.go:166`) and inserts an entry into `s.sessions` (sessionMux-guarded map, unbounded growth) with no rate limit, no auth gate, and no idle/handshake timeout visible in `HandleWebSocket/handleSession`. Any anonymous client can open unlimited concurrent connections.
2. **Forward-risk landmine.** The moment anyone wires `RegisterTool` calls into `server.New's` construction path (the natural next step for an MCP server whose entire purpose is exposing tools to MCP clients), /ws silently becomes an unauthenticated arbitrary-capability-execution surface with **zero warning** at that commit — because the auth gap is upstream of tool registration, not downstream.
3. **CSWSH exposure independent of tool registration.** `CheckOrigin: true` is exactly the anti-pattern OWASP's WebSocket Security Cheat Sheet calls the primary Cross-Site WebSocket Hijacking (CSWSH) vector (§5 below) — any web page, anywhere, can script a same-user browser into opening this socket. Combined with `CORSMiddleware()` (`server.go:628-642`) also

emitting Access-Control-Allow-Origin: * with Access-Control-Allow-Credentials: true (a related but **out-of-scope** finding — CORS wildcard-origin-with-credentials is itself invalid per the Fetch spec and should be tracked separately), this codebase has a consistent pattern of deferring origin validation that this design should not repeat.

4. The core constraint: browsers can't send Authorization on a WS handshake

The native browser WebSocket constructor (`new WebSocket(url[, protocols])`) accepts only a URL and an optional subprotocol list — there is no way for browser JS to attach an `Authorization: Bearer ...` or `x-api-key` header to the handshake GET request. This is a deliberate, unchanged constraint of the WHATWG WebSocket API (confirmed by every source in §5). It is the reason `wireFacadeAuthMiddleware()` (`server.go:571-620`) — which reads `Authorization: Bearer / x-api-key` off `c.GetHeader(...)` — **cannot be reused unmodified for browser-originated /ws connections**, even though it is exactly the right pattern for non-browser clients (see §6, Option B).

Note this constraint does **not** apply to non-browser MCP SDK / CLI clients (Go/Python/Node MCP client libraries, `curl`-style test harnesses) — those can set arbitrary headers on the pre-upgrade HTTP request, so `wireFacadeAuthMiddleware()`-style `Bearer/x-api-key` checking works for them today with no new infrastructure.

5. Research: standard patterns for browser WebSocket auth

Deep research per §11.4.150/§11.4.99 against current (2025-2026) authoritative sources — OWASP's dedicated WebSocket Security Cheat Sheet, `websocket.org`'s guide (Postman-affiliated, actively maintained), and the `websocket`'s Python library's authentication documentation (a widely-cited reference implementation guide) — all independently converge on the same four patterns:

5.1 Ticket / short-lived one-time-token pattern

Client calls an already-authenticated HTTPS endpoint (can reuse existing `Bearer/x-api-key/session` auth) to mint a short-TTL, single-use ticket. Client then opens the WS connection and presents the ticket — **as the first message after connect**

(preferred) or as a query-string param (fallback, with a short TTL to bound exposure).

- OWASP: doesn't name "ticket" explicitly but endorses token-based auth passed "via alternative methods" beyond Authorization headers, and explicitly flags query-string tokens as needing redaction from access logs.
- websocket.org: names first-message auth as keeping tokens "out of logs and browser history entirely," at the cost of requiring an authentication timeout (recommends ~5s) plus IP-level rate limiting on unauthenticated sockets to bound resource exhaustion during that window.
- websockets (readthedocs, Python reference lib, widely cited beyond just Python users): calls first-message token auth "fully reliable and the most secure mechanism," explicitly because it operates at the application layer where the server controls validation timing, versus query-string tokens which "end up in logs, which leaks credentials."
- AWS's own WebSocket-API-Gateway guidance (below) independently recommends the query-string-token variant of this same family as its default, precisely because a Lambda authorizer can reject during the HTTP-upgrade phase before any compute is spent on the connection.

Tradeoff vs HelixCode's existing scheme: requires a NEW minting endpoint + a short-TTL nonce store (single-use tracking) — new infrastructure, but the minting endpoint itself can be gated by the *existing* `wireFacadeAuthMiddleware()`/`HELIX_WIRE_FACADE_API_KEYS` check or the internal-user `authMiddleware()`/`JWT` check, so no new *credential class* is introduced, only a new *short-lived derived* one.

5.2 Sec-WebSocket-Protocol subprotocol-as-token-channel

Browser JS *can* set the Sec-WebSocket-Protocol value even though it can't set Authorization — some implementations (documented AWS API Gateway WebSocket blog posts/writeups) smuggle a JWT or user id through this header, with the server echoing back a recognized subprotocol value to complete the handshake per RFC 6455 §1.9/§4.

- AWS's own more-current guidance (`readysvccloud.io`, AWS Compute Blog) explicitly **recommends the query-string-token approach over subprotocol smuggling** — "the recommendation is using the query string parameter approach because it is straightforward and does not repurpose the Sec-WebSocket-Protocol header."
- Limitations independently confirmed: subprotocol values are still visible to intermediate proxies/logs (no better than query-string on that axis), values are constrained (comma-separated token list per RFC 6455, awkward for anything beyond a short opaque id), and it repurposes a header whose

actual purpose is protocol negotiation — a semantic overload that every source treats as a workaround rather than a recommended pattern.

Verdict: not recommended as primary; documented here because the task brief asked for it explicitly, but no current authoritative source recommends it over the ticket/first-message pattern.

5.3 Cookie-based auth (+ CSWSH risk)

If the WS server shares a domain with the web app, the browser automatically attaches session cookies to the WS handshake (it's still an HTTP GET request). Server validates the cookie exactly like any other HTTP request.

- OWASP: this is the *default* behaviour browsers exhibit and is explicitly named as *why* CSWSH exists — “browsers include cookies in WebSocket handshake requests, making WebSocket applications vulnerable to Cross-Site WebSocket Hijacking (CSWSH)” unless Origin is validated. OWASP’s primary CSWSH defense is **explicit Origin allowlist validation on every handshake** (“Use an allowlist, not a denylist. Avoid wildcards or substring matching.”) — which HelixCode’s `CheckOrigin: true` currently fails outright.
- websocket.org: notes cookie auth requires “zero client-side authentication code” but needs CSRF protection via Origin validation; `SameSite=Strict` cookies won’t even transmit cross-origin, which helps but isn’t sufficient alone (subdomains, and any same-site attacker page, still succeed).
- websockets docs: flags a real gap in cookie auth for multi-tenant/shared parent-domain deployments — “the cookie would be shared with all subdomains of the parent domain... for a cookie containing credentials, this is unacceptable” in that topology.

Tradeoff vs HelixCode’s existing scheme: HelixCode’s internal-user auth today is Bearer-JWT-in-Authorization header (`authMiddleware()`, `server.go:488-543`), **not** cookie-based — adopting cookie auth for `/ws` would introduce a parallel credential-delivery mechanism not used anywhere else in the codebase, plus require confirming the frontend actually performs a cookie-issuing login flow today (**UNCONFIRMED** — see §9).

5.4 Query-string token (as its own option, not just ticket-pattern fallback)

Every source above converges on the same tradeoff independently: fast rejection during the HTTP-upgrade phase (before any WS-frame compute is spent) vs the token appearing in access logs / proxy logs / browser history / `Referer` headers on outbound requests from the page. Universal mitigation across sources: make it short-TTL (websocket.org suggests 5-15 minutes) and single-use, and never put a

long-lived credential (e.g. the raw `HELIX_WIRE_FACADE_API_KEYS` value) directly in a query string.

6. Recommended primary option + implementation sketch

Recommended: Option A — Ticket pattern for browser clients, layered on top of a direct Bearer/x-api-key check for non-browser clients, plus a mandatory Origin-allowlist fix regardless of which auth option is chosen.

This is because (a) it's the pattern every current authoritative source converges on for genuine browser JS clients where Authorization cannot be set at all, (b) it reuses HelixCode's *existing* credential class (`HELIX_WIRE_FACADE_API_KEYS` or the internal JWT) to gate the ticket-minting endpoint rather than inventing a new standing secret, and (c) it composes cleanly with the non-browser lane (Option B, §8) which can ship independently and immediately at near-zero risk.

6.1 Sketch — new pieces (design only, not implemented)

1. **New minting endpoint** `POST /api/v1/ws-ticket`, gated by `s.wireFacadeAuthMiddleware()` (or `s.authMiddleware()` for internal-user sessions — whichever the caller already authenticates with). Handler mints a random 256-bit nonce, signs `{nonce, exp: now+30s, aud: "ws", sub: <caller id>}` (reuse `s.auth`'s existing JWT signing machinery with a distinct, short `TokenExpiry` override — do **not** reuse the long-lived session JWT verbatim), and stores the nonce as *unconsumed* — in Redis (`s.redis`, already wired into `Server`) with a 30-60s TTL if `s.redis` is enabled, else an in-process `sync.Map` with a background sweep (mirrors the `s.sessions` map pattern already in `mcp.MCPServer`) when Redis is disabled. Returns `{"ticket": "<compact-token>"}`.
2. **/ws handshake gate, before Upgrade()**:
 - If the pre-upgrade request carries `Authorization: Bearer` or `x-api-key` matching `HELIX_WIRE_FACADE_API_KEYS` (non-browser lane, Option B) → accept immediately, same fail-closed semantics as `wireFacadeAuthMiddleware()`.
 - Else, require the connection's **first WebSocket message** (not a query param, to keep the ticket out of access logs per §5.1/§5.4) to be an auth frame `{"type": "auth", "ticket": "<ticket>"}` within a bounded timeout (5s, mirroring `websocket.org`'s guidance) — validate signature + expiry + single-use (atomically mark consumed in the Redis/`sync.Map` store; a second use of the same ticket is rejected). On success, proceed into the existing `handleSession` loop. On failure or timeout, close with WS close code 1008 (Policy Violation) and log the rejection (never log

the ticket value itself, per OWASP’s “avoid logging sensitive data” guidance).

3. **Origin allowlist fix (mandatory regardless of chosen option):** replace `internal/mcp/server.go:104-107`’s `CheckOrigin: func(r *http.Request) bool { return true }` with an explicit allowlist sourced from `config` (new `cfg.Auth.WSAllowedOrigins []string`, analogous to `WireFacadeAPIKeys`), defaulting to same-origin-only when unset — never a wildcard.
4. **Idle/unauthenticated-socket timeout, independent of the auth outcome:** set a read deadline on the connection immediately after `Upgrade()` (`conn.SetReadDeadline(time.Now().Add(5*time.Second))`) so a client that never sends the auth frame at all cannot hold a goroutine + session-map entry open indefinitely — closes the DoS gap identified in §3.1 independently of whether auth itself is bypassed.

6.2 Where this lives in the existing code (file:line references for the follow-up implementation task)

- `helix_code/internal/config/config.go` — extend `AuthConfig` (currently ends around line 109 with `WireFacadeAPIKeys`) with `WSAllowedOrigins` and (if the ticket’s own short-TTL signing key should be distinct from `JWTSecret`) an optional `WSTicketSigningKey`.
- `helix_code/internal/server/server.go:430` — replace the bare `s.router.GET("/ws", s.handleWebSocket)` registration with a new `s.wsAuthMiddleware()` (new middleware, sibling to `wireFacadeAuthMiddleware` at line 571) that implements the pre-upgrade Bearer/x-api-key check (Option B) and, if absent, passes a flag/context value telling `handleWebSocket` to require the post-upgrade first-message ticket check.
- `helix_code/internal/mcp/server.go:100-112` (`NewMCPServer`) — accept an `allowedOrigins []string` (or a `CheckOrigin func(*http.Request) bool`) constructor parameter instead of hardcoding `return true`, and `internal/mcp/server.go:142` (`HandleWebSocket`) — accept the ticket-validation callback / set the post-upgrade read deadline described in §6.1.3-4.
- New file `helix_code/internal/server/ws_ticket.go` (or similar) — the `POST /api/v1/ws-ticket` handler + nonce store, following the existing file-per-facade convention (`wire_facade.go` is the precedent).

7. Regression-guard test plan (§11.4.135) + RED plan (§11.4.115)

Mirrors the DZ-05 precedent explicitly cited in `server.go`'s own comments (`wire_facade_auth_test.go`, `wire_facade_live_e2e_test.go`) — real gin router via `server.New( . . . )`/the actual route table, real gorilla/websocket client dial, no mocks (§11.4.27(A) unit-test-only exception does not apply here; this must be an integration-style test against the real router per CONST-050(A)).

7.1 RED (pre-fix, §11.4.115 polarity, RED_MODE=1 default)

`TestWebSocketUnauthenticatedHandshake_RED` — spins `httptest.NewServer` wrapping the real router, dials `ws://<addr>/ws` with `websocket.DefaultDialer.Dial( . . . )` and **no** Authorization header, no cookie, no ticket. **On the current (pre-fix) code this MUST assert the dial SUCCEEDS** (101 Switching Protocols, no error) — this is the captured, reproducing proof of the defect described in §1, run against the actual broken artifact, not a synthetic scenario.

7.2 GREEN (post-fix, RED_MODE=0, same test source, polarity flipped)

Same test, same assertion site, flipped expectation: dial with no credential **MUST fail** — either the HTTP upgrade itself returns 401 (non-browser lane, if the fix rejects at `wsAuthMiddleware( )` before `Upgrade( )` even for a credential-less request) or the WS connection opens and is immediately closed with code 1008 within the 5s timeout (browser lane, ticket never sent). Both are acceptable “rejected” shapes; the test asserts one of them, not a specific status code, to avoid over-coupling to option A vs B's exact mechanics.

7.3 Standing regression-guard suite (registered same commit as the fix, §11.4.135)

Case	Expected
No credential, no ticket	Rejected (§7.2)
Valid HELIX_WIRE_FACADE_API_KEYS Bearer/x-api-key on handshake	Accepted (non-browser lane)
Valid, unexpired, unconsumed ticket as first message	Accepted (browser lane)
Expired ticket (mint, sleep past TTL, then send)	Rejected

Case	Expected
Already-consumed ticket (use once successfully, dial again with same ticket)	Rejected — proves single-use
Valid ticket but Origin header not in allowlist	Rejected — proves Origin check is enforced independently of ticket validity
Connection opened, no auth frame sent within 5s	Server-initiated close within timeout — proves no unbounded resource hold
/api/v1/ws-ticket called with no/invalid Bearer/x-api-key	401, mirrors wireFacadeAuthMiddleware fail-closed semantics

Every PASS in this suite must carry the actual dial/response evidence (§11.4.5/§11.4.69) — a “no error returned” check without inspecting the actual close code / HTTP status is not sufficient per the §11.4 anti-bluff covenant.

8. Operator decision (§11.4.66)

Minimum-viable operator input, framed as options:

Option A — Ticket pattern (browser lane) + Bearer/x-api-key (non-browser lane) + Origin-allowlist fix. [Recommended] Trade-off: most complete fix (closes browser CSWSH gap, non-browser gap, and the DoS/idle-socket gap in one pass); requires new minting endpoint + nonce store + two new test files; ~1-2 days of implementation + test work. *After this answer:* implement §6’s sketch in full, land the regression suite in §7.3, verify against a live /api/v1/ws-ticket → /ws round trip from a real browser-equivalent client (per §11.4.98 full automation — no manual browser click-through as the only evidence).

Option B — Bearer/x-api-key on the pre-upgrade handshake only (mirror wireFacadeAuthMiddleware, reuse HELIX_WIRE_FACADE_API_KEYS verbatim), plus the Origin-allowlist fix. No ticket infrastructure. Trade-off: smallest, fastest change (essentially the same one-line pattern already applied to /v1/chat/completions//v1/messages); closes the non-browser-client gap and the CSWSH/Origin gap immediately; **does not close the gap for genuine browser-JS clients** (the static web UI at /), because browser WebSocket () cannot set these headers — if nothing in HelixCode’s frontend currently opens /ws from browser JS, this option may be sufficient as a complete fix; if it does (or will), this option leaves that path unauthenticated or breaks it outright once the middleware

is strict. *After this answer:* implement the middleware + Origin fix only, skip the ticket endpoint, and the doc's §7 suite drops the ticket-specific rows.

Option C — Cookie-based auth (httpOnly, SameSite=Strict) + strict Origin allowlist, contingent on confirming the frontend already performs cookie-issuing login. Trade-off: zero client-side WS-specific code once wired; but introduces a credential-delivery mechanism (cookies) not used anywhere else in this codebase today (everything else is Bearer-JWT-in-header), and does not help non-browser MCP SDK clients on its own (would still need Option B layered on top for those). *After this answer:* first confirm (read-only) whether `s.auth`'s login flow issues a session cookie anywhere today; if not, this option requires standing up new cookie-issuing infrastructure before `/ws` work can start, materially larger than A or B.

Option D — Ship Option B now (fast, closes the biggest and most certain gap: any-origin + any-client unauthenticated access), track Option A's ticket lane as a fast-follow only if/when a genuine browser-JS `/ws` client is confirmed to exist or is planned. Trade-off: fastest path to closing the confirmed, currently-exploitable gap (§1-§3) without over-building for a browser use case that may not exist yet; requires a follow-up confirmation step (`grep web/frontend/**` for any new `WebSocket` (call) before deciding whether Option A is ever needed at all.

9. Honest concerns / UNCONFIRMED items

- **UNCONFIRMED: does any current HelixCode client actually open `/ws` from browser JS today?** This design assumed yes (the static web UI served at `server.go:433-434`, `/static + /`) is the plausible consumer, but this agent did not `grep web/frontend/**` for a live new `WebSocket( . . . )` call site within this task's scope (read-only inspection was scoped to the server-side handler + auth patterns per the task brief). **This is the single highest-leverage fact to confirm before choosing between Option A/D and Option B** — if no browser client exists yet, Option B alone may be a complete, much cheaper fix.
- **UNCONFIRMED: does `s.auth`/the login flow issue any cookie today, or is every session strictly Bearer-JWT-in-header?** Needed to evaluate whether Option C is even feasible without new infra. Not verified in this pass.
- **UNCONFIRMED:** whether Redis (`s.redis`) is reliably enabled in every deployment profile this server ships — the ticket nonce store's HA/TTL behavior differs meaningfully between the Redis-backed and in-process `sync.Map` fallback paths; the in-process fallback does not survive a process restart or work across multiple server replicas behind a load balancer (a ticket minted on replica A would not be recognized as consumed by replica B). If HelixCode ever runs multi-replica, Option A's nonce store **MUST** be the

Redis path, not an in-process map — flagging this now so the follow-up implementation task doesn't silently pick the simpler-but-wrong variant.

- **Related but explicitly out-of-scope finding:** `CORSMiddleware()` (`server.go:628-642`) emits `Access-Control-Allow-Origin: *` together with `Access-Control-Allow-Credentials: true` — invalid per the Fetch spec (a wildcard origin is not supposed to be paired with credentials) and a separate, pre-existing security smell independent of /ws. Not touched by this design; recommend a separate workable item.
- No secrets were read, generated, or logged in the course of this research (§11.4.10 compliant) — no `.env` file was opened.

Sources verified

- OWASP WebSocket Security Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html (accessed 2026-07-08)
- websocket.org — WebSocket Authentication guide — <https://websocket.org/guides/authentication/> (accessed 2026-07-08)
- websocket.org — WebSocket Security guide (CSWSH, Origin, rate limiting) — <https://websocket.org/guides/security/> (accessed 2026-07-08, search-result summary; not separately WebFetched)
- websockets (Python reference library) — Authentication topic guide — <https://websockets.readthedocs.io/en/stable/topics/authentication.html> (accessed 2026-07-08)
- AWS — “Intro to AWS WebSockets Part Two: Auth” (Ready, Set, Cloud!) — <https://www.readysetcloud.io/blog/allen.helton/intro-to-aws-websockets-part-two/> (accessed 2026-07-08, search-result summary)
- AWS Compute Blog — “Managing sessions of anonymous users in WebSocket API-based applications” — <https://aws.amazon.com/blogs/compute/managing-sessions-of-anonymous-users-in-websocket-api-based-applications/> (accessed 2026-07-08, search-result summary)
- RFC 6455 (The WebSocket Protocol), §1.9/§4 — `Sec-WebSocket-Protocol` semantics (referenced, not re-fetched this session — well-established IETF standard)

No source contradicted another on the core recommendation (ticket/first-message pattern preferred over query-string or subprotocol-smuggling for browser clients where log/history exposure matters); AWS's own more-recent guidance independently agrees query-string tokens are preferable to `Sec-WebSocket-Protocol` smuggling when a query-string-class approach is used at all.